ArrayList en Java

Programación II — Licenciatura en Ciencias de la Computación

Autores:

Lautaro Larosa
Gabriel Lopez Romero

4 de septiembre de 2026

Índice

1. Introducción **2**

2. Estructura del Proyecto **2**

3. Código **2**
3.1. Clase Empleado 2
3.2. Clase ActividadArraylist 3

4. Capturas de Pantalla de la Ejecución **6**
4.1. Carga de Empleados y Menú Principal 6
4.2. Búsqueda Individual de Empleados 7
4.3. Listado Completo en Orden Normal 8
4.4. Listado Completo en Orden Invertido 8

5. Conclusión **9**

1. Introducción

El presente trabajo consiste en el desarrollo de una aplicación de consola en Java que implementa la estructura de datos dinámica `ArrayList` para gestionar registros de empleados. Se contempla la carga interactiva de datos, búsqueda individual por múltiples criterios (posición, nombre, apellido y legajo) y el listado de elementos tanto en sentido secuencial como invertido.

2. Estructura del Proyecto

El proyecto se organiza en dos clases principales bajo el paquete `actividadarraylist`:

- **Empleado:** Clase modelo con los atributos encapsulados `nombre`, `apellido` y `legajo`, constructores, getters, setters y sobreescritura de `toString()`.
- **ActividadArraylist:** Clase ejecutable con el menú interactivo, control de flujo con bucle `do-while`, métodos auxiliares de búsqueda y visualización.

3. Código

3.1. Clase Empleado

```java
package actividadarraylist;

public class Empleado {
    private String nombre;
    private String apellido;
    private int legajo;

    public Empleado() {
    }

    public Empleado(String nombre, String apellido, int legajo) {
        this.nombre = nombre;
        this.apellido = apellido;
        this.legajo = legajo;
    }

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public String getApellido() {
        return apellido;
    }

    public void setApellido(String apellido) {
        this.apellido = apellido;
    }

    public int getLegajo() {
        return legajo;
    }

    public void setLegajo(int legajo) {
        this.legajo = legajo;
```

```
    }

    @Override
    public String toString() {
        return "Legajo: " + legajo + " | Nombre: " + nombre + " | Apellido: " +
    apellido;
    }
}
```

Listing 1: Empleado.java

3.2. Clase ActividadArraylist

```
package actividadarraylist;

import java.util.ArrayList;
import java.util.Iterator;
import java.util.Scanner;

public class ActividadArraylist {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        ArrayList<Empleado> empleados = new ArrayList<>();
        int opcion;

        do {
            System.out.println("\n--- Menu Principal ---");
            System.out.println("1. Agregar empleado");
            System.out.println("2. Mostrar un empleado especifico");
            System.out.println("3. Mostrar lista completa (normal)");
            System.out.println("4. Mostrar lista completa (invertida)");
            System.out.println("5. Salir");
            System.out.print("Seleccione una opcion: ");

            // Limpieza de buffer del Scanner tras leer valores numericos.
            opcion = sc.nextInt();
            sc.nextLine();

            switch (opcion) {
                case 1 -> agregarEmpleado(sc, empleados);
                case 2 -> buscarEmpleado(sc, empleados);
                case 3 -> mostrarNormal(empleados);
                case 4 -> mostrarInvertido(empleados);
                case 5 -> System.out.println("Fin del programa.");
                default -> System.out.println("Opcion no valida.");
            }
        } while (opcion != 5);
    }

    private static void agregarEmpleado(Scanner sc, ArrayList<Empleado>
    empleados) {
        System.out.print("Ingrese nombre: ");
        String nombre = sc.nextLine();
        System.out.print("Ingrese apellido: ");
        String apellido = sc.nextLine();
        System.out.print("Ingrese legajo: ");
        int legajo = sc.nextInt();
        sc.nextLine();

        empleados.add(new Empleado(nombre, apellido, legajo));
        System.out.println("Empleado agregado.");
```

```
49      }
50
51      private static void buscarEmpleado(Scanner sc, ArrayList<Empleado> empleados
        ) {
52          if (empleados.isEmpty()) {
53              System.out.println("No hay empleados registrados.");
54              return;
55          }
56
57          System.out.println("\nBuscar por:");
58          System.out.println("1. Posicion en lista");
59          System.out.println("2. Nombre");
60          System.out.println("3. Apellido");
61          System.out.println("4. Legajo");
62          System.out.print("Opcion: ");
63          int criterio = sc.nextInt();
64          sc.nextLine();
65
66          switch (criterio) {
67              case 1 -> {
68                  System.out.print("Ingrese indice (0 a " + (empleados.size() - 1)
         + "): ");
69                  int indice = sc.nextInt();
70                  sc.nextLine();
71                  if (indice >= 0 && indice < empleados.size()) {
72                      System.out.println(empleados.get(indice));
73                  } else {
74                      System.out.println("Indice fuera de rango.");
75                  }
76              }
77              case 2 -> {
78                  System.out.print("Ingrese nombre: ");
79                  String nom = sc.nextLine();
80                  Empleado emp = buscarPorNombre(empleados, nom);
81                  System.out.println(emp != null ? emp : "Empleado no encontrado."
        );
82              }
83              case 3 -> {
84                  System.out.print("Ingrese apellido: ");
85                  String ape = sc.nextLine();
86                  Empleado emp = buscarPorApellido(empleados, ape);
87                  System.out.println(emp != null ? emp : "Empleado no encontrado."
        );
88              }
89              case 4 -> {
90                  System.out.print("Ingrese legajo: ");
91                  int leg = sc.nextInt();
92                  sc.nextLine();
93                  Empleado emp = buscarPorLegajo(empleados, leg);
94                  System.out.println(emp != null ? emp : "Empleado no encontrado."
        );
95              }
96              default -> System.out.println("Criterio no valido.");
97          }
98      }
99
100     private static Empleado buscarPorNombre(ArrayList<Empleado> empleados,
        String nombre) {
101         for (Empleado e : empleados) {
102             if (e.getNombre().equalsIgnoreCase(nombre)) {
103                 return e;
104             }
105         }
```

```
106         return null;
107     }
108 
109     private static Empleado buscarPorApellido(ArrayList<Empleado> empleados,
    String apellido) {
110         for (Empleado e : empleados) {
111             if (e.getApellido().equalsIgnoreCase(apellido)) {
112                 return e;
113             }
114         }
115         return null;
116     }
117 
118     private static Empleado buscarPorLegajo(ArrayList<Empleado> empleados, int
    legajo) {
119         for (Empleado e : empleados) {
120             if (e.getLegajo() == legajo) {
121                 return e;
122             }
123         }
124         return null;
125     }
126 
127     private static void mostrarNormal(ArrayList<Empleado> empleados) {
128         if (empleados.isEmpty()) {
129             System.out.println("No hay empleados registrados.");
130             return;
131         }
132         System.out.println("\nLista de empleados:");
133         Iterator<Empleado> it = empleados.iterator();
134         while (it.hasNext()) {
135             System.out.println(it.next());
136         }
137     }
138 
139     private static void mostrarInvertido(ArrayList<Empleado> empleados) {
140         if (empleados.isEmpty()) {
141             System.out.println("No hay empleados registrados.");
142             return;
143         }
144         // Recorrido inverso con indices decrecientes.
145         System.out.println("\nLista de empleados (orden invertido):");
146         for (int i = empleados.size() - 1; i >= 0; i--) {
147             System.out.println(empleados.get(i));
148         }
149     }
150 }
```

Listing 2: ActividadArraylist.java

4. Capturas de Pantalla de la Ejecución

4.1. Carga de Empleados y Menú Principal

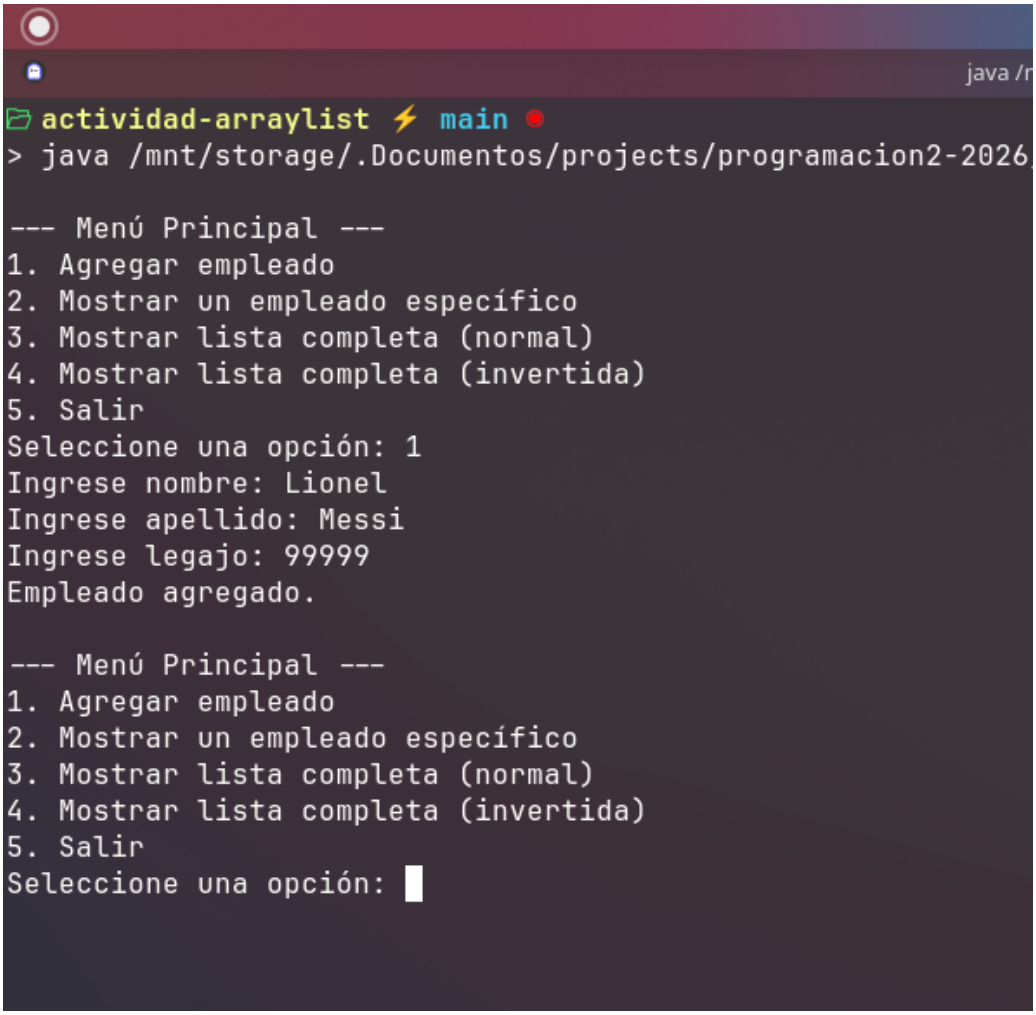


Figura 1: Ejecución del menú principal y registro de nuevos empleados.

4.2. Búsqueda Individual de Empleados

```
--- Menú Principal ---
1. Agregar empleado
2. Mostrar un empleado específico
3. Mostrar lista completa (normal)
4. Mostrar lista completa (invertida)
5. Salir
Seleccione una opción: 2

Buscar por:
1. Posición en lista
2. Nombre
3. Apellido
4. Legajo
Opción: 4
Ingrese legajo: 99999
Legajo: 99999 | Nombre: Lionel | Apellido: Messi

--- Menú Principal ---
1. Agregar empleado
2. Mostrar un empleado específico
3. Mostrar lista completa (normal)
4. Mostrar lista completa (invertida)
5. Salir
Seleccione una opción:
```

Figura 2: Resultados de consultas puntuales por diferentes criterios.

4.3. Listado Completo en Orden Normal

```
--- Menú Principal ---
1. Agregar empleado
2. Mostrar un empleado específico
3. Mostrar lista completa (normal)
4. Mostrar lista completa (invertida)
5. Salir
Seleccione una opción: 3

Lista de empleados:
Legajo: 99999 | Nombre: Lionel | Apellido: Messi
Legajo: 77777 | Nombre: Cristiano | Apellido: Ronaldo
Legajo: 11111 | Nombre: Neymar | Apellido: Junior
Legajo: 11112 | Nombre: Arjen | Apellido: Robben
Legajo: 12312 | Nombre: Harry | Apellido: Kane

--- Menú Principal ---
1. Agregar empleado
2. Mostrar un empleado específico
3. Mostrar lista completa (normal)
4. Mostrar lista completa (invertida)
5. Salir
Seleccione una opción:
```

Figura 3: Visualización secuencial de todos los registros cargados.

4.4. Listado Completo en Orden Invertido

```
--- Menú Principal ---
1. Agregar empleado
2. Mostrar un empleado específico
3. Mostrar lista completa (normal)
4. Mostrar lista completa (invertida)
5. Salir
Seleccione una opción: 4

Lista de empleados (orden invertido):
Legajo: 12312 | Nombre: Harry | Apellido: Kane
Legajo: 11112 | Nombre: Arjen | Apellido: Robben
Legajo: 11111 | Nombre: Neymar | Apellido: Junior
Legajo: 77777 | Nombre: Cristiano | Apellido: Ronaldo
Legajo: 99999 | Nombre: Lionel | Apellido: Messi
```

Figura 4: Visualización de registros recorridos desde el último hacia el primero.

5. Conclusión

El uso de `ArrayList` permitió gestionar una colección dinámica de objetos de forma eficiente. La implementación del patrón `Iterator` facilitó el recorrido secuencial estándar, mientras que el acceso aleatorio mediante `get(int)` permitió realizar el recorrido inverso y búsquedas por índice de manera directa.